# Advanced Strings

## Eertree, suffix trees, Lyndon words, periodicity and runs

# Learning goals

- Build the **palindromic tree (eertree)** online in O(n) and count palindrome occurrences
- Use **series links** for O(n log n) palindromic factorisation DP
- Obtain a **suffix tree** from a suffix automaton and use it for LCP / substring queries
- Know **Lyndon words**, the Chen–Fox–Lyndon theorem, and **Duval's algorithm**
- Reason with periods: border duality, **Fine–Wilf**, and the **Runs theorem**

# Where we are

- Week 28–29: hashing, KMP, Z, tries, Aho–Corasick, SA intro
- Regional-09: Manacher, palindromes, string DP
- Nationals-06: suffix array + LCP, **suffix automaton**
- **Today:** the structures that appear when those are not enough
- Theme: strings have rigid combinatorial structure — *few* distinct palindromes, *few* runs, periods in *arithmetic progressions*

# Fact: few distinct palindromes

**Lemma.** A string of length n has at most n distinct non-empty palindromic substrings.

Proof: appending character c creates at most one *new* palindrome — the longest palindromic suffix P of s·c. Any shorter palindromic suffix Q of P is also a prefix of P (palindrome), so it occurred earlier. ∎

⇒ A data structure with **one node per distinct palindrome** has O(n) size: the eertree.

# The eertree

- Two roots: **−1** (imaginary, length −1) and **0** (empty string)
- Node = distinct palindrome; edge `v –c→ u` means u = c·v·c
- Children of root −1 are the single characters
- **Suffix link** of v: longest proper palindromic suffix of v
- `last` = node of the longest palindromic suffix of the current prefix

```
len -1 ──a──▶ "a" ──b──▶ "bab"
len  0 ──a──▶ "aa"
link("bab") = "b",  link("a") = empty,  link(empty) = -1
```

# Adding a character

To append s[i] = c:

1. From `last`, follow suffix links until node X with s[i − |X| − 1] = c
2. If X has edge c: `last = X.next[c]`, done (no new palindrome)
3. Else create node cXc with length |X| + 2
4. Its suffix link: continue from link(X) to the next Y with s[i − |Y| − 1] = c, link = Y.next[c] (or the empty node if length 1)

Amortised O(1) per character: the depth of `last` in the link tree grows by ≤ 1 per step (same argument as KMP).

# Eertree code

```
struct Eertree {
    struct Node { int len, link, cnt, diff, slink; array<int, 26> nxt; };
    vector<Node> t; string s; int last;
    int newNode(int len, int link) { t.push_back({len, link, 0, 0, 0, {}}); return (int)t.size() - 1; }
    Eertree() { newNode(-1, 0); newNode(0, 0); last = 1; }
    int getLink(int v, int i) const {
        while (i - 1 - t[v].len < 0 || s[i - 1 - t[v].len] != s[i]) v = t[v].link;
        return v;
    }
    bool add(char ch) {
        s.push_back(ch); int i = (int)s.size() - 1, c = ch - 'a';
        int cur = getLink(last, i); bool created = false;
        if (!t[cur].nxt[c]) {
            int len = t[cur].len + 2;
            int link = len == 1 ? 1 : t[getLink(t[cur].link, i)].nxt[c];   // BEFORE adding the edge
            int v = newNode(len, link);
            t[v].diff = len - t[link].len;
            t[v].slink = t[v].diff == t[link].diff ? t[link].slink : link;
            t[cur].nxt[c] = v; created = true;
        }
        last = t[cur].nxt[c]; t[last].cnt++;
        return created;
    }
};
```

# Counting occurrences

- `cnt[v]` after construction = #positions where v is the **longest** palindromic suffix
- Every palindromic suffix of a prefix is on the suffix-link chain of `last`
- So occ(v) = Σ cnt over the link-subtree of v
- Links always point to **older** nodes ⇒ one reverse pass:

```
for (int v = t.size() - 1; v >= 2; v--) t[t[v].link].cnt += t[v].cnt;
```

APIO 2014 *Palindromes*: max over v of len(v) · occ(v) — 10 lines on top of the eertree.

# Palindromic suffixes form O(log n) progressions

**Lemma.** The lengths of the palindromic suffixes of any string can be partitioned into O(log n) **arithmetic progressions**.

- If P is a palindrome and Q a palindromic suffix with $|Q| \geq |P|/2$, then $|P| - |Q|$ is a period of P
- diff(v) = len(v) − len(link(v)); diff is non-increasing along links
- Each time diff changes, the length at least halves ⇒ O(log n) groups
- **Series link** slink(v) = first ancestor with a different diff

# Palindromic factorisation in O(n log n)

dp[i] = min #palindromes that s[0, i) splits into.

```
dp[0] = 0;
for (int i = 1; i <= n; i++) {
    T.add(s[i - 1]);
    for (int v = T.last; T.t[v].len > 0; v = T.t[v].slink) {
        series[v] = dp[i - (T.t[T.t[v].slink].len + T.t[v].diff)];
        if (T.t[v].diff == T.t[T.t[v].link].diff)
            series[v] = min(series[v], series[T.t[v].link]);
        dp[i] = min(dp[i], series[v] + 1);
    }
}
```

`series[v]` caches the min over a whole progression; it was last updated at position i – diff(v). Works for any associative aggregate (count, min, sum) — CF 932G, CF 906E.

# Suffix trees

- Compressed trie of all suffixes of s (add `$` to make every suffix a leaf)
- ≤ 2n nodes; edge labels stored as (start, length) in s
- Node depth = length of the string it spells
- LCP(suffix i, suffix j) = depth(LCA(leaf i, leaf j))
- Distinct substrings = Σ edge lengths
- Ukkonen builds it online in O(n), but it is long and fragile under contest pressure

# Suffix tree = link tree of SAM(reverse s)

- SAM state v of t = reverse(s): the substrings with the same endpos-set, lengths (len(link), len(v)]
- Reversed back into s, they are the **prefixes** of one string w_v of lengths (len(link), len(v)]
- Prefixes of the same string ⇒ a path in a trie of right-extensions ⇒ the suffix tree of s
- Edge link(v) → v is labelled w_v[len(link) .. len(v))
- w_v starts in s at n − 1 − firstEnd(v), where firstEnd = first end position in t

O(n · log σ) with `map`, and you already know the SAM from nationals-06.

# Suffix tree code

```
// after building the SAM of t = reverse(s) with firstEnd[v] recorded on creation
// (a clone copies firstEnd from the state it clones):
for (int v = 1; v < (int)st.size(); v++) {
    ch[st[v].link].push_back(v);
    start[v] = n - 1 - st[v].firstEnd;        // w_v = s[start[v], start[v] + len[v])
}
// edge link(v) -> v spells s[start[v] + len[link(v)], start[v] + len[v])
```

- Sort children by first character of the edge for lexicographic DFS
- Leaves (and some internal nodes) correspond to suffixes of s
- Alternative: SA + LCP array ⇒ "virtual" suffix tree as the Cartesian tree of LCP (CF 1073G)

# Lyndon words

A non-empty string w is **Lyndon** if it is strictly smaller than all its proper rotations (equivalently: than all its proper suffixes).

- `a`, `ab`, `aab`, `abb`, `aabab` are Lyndon; `aa`, `abab`, `ba` are not
- A Lyndon word has no non-trivial border
- If u < v are Lyndon, then uv is Lyndon

**Chen–Fox–Lyndon:** every string factors **uniquely** as s = $w_1$ $w_2$ … w_k with w_i Lyndon and $w_1 \geq w_2 \geq \ldots \geq$ w_k.

# Duval's algorithm

Maintain $s = s_1\, s_2\, s_3$ where $s_1$ is factored, $s_2 = (u)^m\, u'$ is a "pre-Lyndon" block ($u$ Lyndon, $u'$ a proper prefix of $u$), and $j$ scans $s_3$.

- `s[j] == s[k]`: periodic continuation, k++
- `s[j] > s[k]`: $s_2$ + s[j] becomes one Lyndon word, k = i
- `s[j] < s[k]`: emit the copies of u, restart after them

O(n) time, O(1) extra memory, fewer than 4n character comparisons.

# Duval and minimal rotation code

```
vector<pair<int,int>> duval(const string& s) {          // (start, length) per factor
    vector<pair<int,int>> res; int n = s.size(), i = 0;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) { k = (s[k] < s[j]) ? i : k + 1; j++; }
        while (i <= k) { res.push_back({i, j - k}); i += j - k; }
    }
    return res;
}
int minRotation(string s) {                              // start of least rotation
    int n = s.size(), i = 0, ans = 0; s += s;
    while (i < n) {
        ans = i; int j = i + 1, k = i;
        while (j < 2 * n && s[k] <= s[j]) { k = (s[k] < s[j]) ? i : k + 1; j++; }
        while (i <= k) i += j - k;
    }
    return ans;
}
```

# Periods and borders

- p is a **period** of s ⇔ s[i] = s[i + p] for all valid i ⇔ s has a **border** of length n − p
- All borders: π[n−1], π[π[n−1]−1], … (prefix function)
- **Fine–Wilf:** if p and q are periods and p + q − gcd(p, q) ≤ n, then gcd(p, q) is a period
- **Consequence:** the periods ≤ n/2 are all multiples of the smallest period
- The borders of s form O(log n) arithmetic progressions (same structure as palindromic suffixes)

Classic use: "is s a k-th power?" ⇔ smallest period p divides n and n/p is a multiple of k.

# Runs

A **run** is a maximal periodic substring s[l, r) with smallest period p and r – l ≥ 2p (maximal: cannot extend left or right with the same period).

- `aabaabaab` has runs `aa` (p=1) ×3 and `aabaabaab` (p=3)
- **Runs theorem** (Bannai et al., 2017): fewer than **n** runs; Σ exponents < 3n
- Every square, tandem repeat, and repetition lies inside a run

# Lyndon roots ⇒ an algorithm

For run (l, r, p) pick the order $\prec \in \{<, >\}$ on letters such that the character after the run is $\prec$-smaller than its counterpart one period earlier.

- Then every $\prec$-Lyndon rotation of the period inside the run, starting at i, is the **longest $\prec$-Lyndon word starting at i**
- So: for both orders, for each i, take j = i + Lyn(i) and test whether the period p = j − i extends: L = LCS(i, j), R = LCP(i, j); run if L + R ≥ p
- Lyn(i) = next smaller suffix to the right (monotone stack over suffix comparisons)

# Runs code

```
for (int inv = 0; inv < 2; inv++) {
    auto less = [&](int i, int j) {                  // suffix i < suffix j in order inv
        int l = lcp(i, j);
        if (i + l == n) return true;
        if (j + l == n) return false;
        return inv ? s[i + l] > s[j + l] : s[i + l] < s[j + l];
    };
    vector<int> stk;
    for (int i = n - 1; i >= 0; i--) {
        while (!stk.empty() && !less(stk.back(), i)) stk.pop_back();
        int j = stk.empty() ? n : stk.back();
        int p = j - i, L = lcs(i, j), R = j < n ? lcp(i, j) : 0;
        if (j < n && L + R >= p) res.insert({i - L, j + R, p});
        stk.push_back(i);
    }
}
```

With hashing + binary search for lcp/lcs: $O(n \log^2 n)$; with SA + sparse table: $O(n \log n)$.

# Applications of periodicity

- Count distinct squares, count all tandem repeats (sum over runs)
- "Longest substring that is a k-th power": max over runs of $\lfloor (r - l)/p \rfloor$
- Pattern matching with a periodic pattern: occurrences form arithmetic progressions
- CF 526D *Om Nom and Necklace*: prefixes of the form (AB)^k A via the smallest period
- Compressing a string as A^k: smallest period from the prefix function

# Pitfalls

- Eertree: computing the new node's link **after** inserting the edge ⇒ a node links to itself
- Eertree: `vector` reallocation invalidates `Node&` references — use indices
- Occurrence counts: propagate in reverse **creation** order, not by length
- SAM → suffix tree: clones must inherit `firstEnd` from the cloned state
- Duval: the inner `while (i <= k)` emits several equal factors — do not forget repeated factors
- Runs: deduplicate (both orders may report the same run)

# Practice problems

- APIO 2014 *Palindromes* (oj.uz `APIO14_palindrome`) — eertree occurrences
- Timus **1960** *Palindromes and Super Abilities* — eertree online
- CF **17E** *Palisection* — counting crossing palindromes
- CF **932G** *Palindrome Partition*, CF **906E** *Reverses* — series links
- UVa **719** *Glass Beads*, SPOJ **MINMOVE** — minimal rotation
- Library Checker *Lyndon Factorization*, *Run Enumerate*
- CF **594E** *Cutting the Line* — Lyndon factorisation, hard
- CF **526D** *Om Nom and Necklace*, CF **432D** *Prefixes and Suffixes* — periods/borders
- CF **1073G** *Yet Another LCP Problem* — suffix tree structure via SA + LCP

## Summary

- ≤ n distinct palindromes ⇒ eertree, O(n); series links ⇒ O(log n) progressions
- Suffix tree = link tree of the SAM of the reversed string
- Lyndon factorisation is unique; Duval finds it in O(n) and gives minimal rotations
- Periods: border duality, Fine–Wilf; fewer than n runs, found via Lyndon roots

**Next:** Lecture 9 — SOS DP, broken profile, Aliens trick, slope trick